

[CSE299(1) - Group#2] Weekly Report: Classroom Booking System - Priority-Based Faculty Allocation

Tawhid Hasan (2413054042), Mohammad Tasvirul Hasan Riyad (2413692042),
Md. Tanvir Islam (2312491042), Md. Tayasshuk Imam (2411600042)
{tawhid.hasan.241, tasvirul.riyad.241, tanvir.islam2, tayasshuk.imam.241}@northsouth.edu
Summer 2026

I. WEEKLY REPORT 1

A. Follow-up from last week

Initial report. No prior tasks to follow up on.

B. Work narrative of the current week

- **Tasks completed:** Finalized the core system architecture using React 19.2.7 for the frontend UI layer [7], Node.js for the server API logic [4], and PostgreSQL 18 for backend relational storage [6], [11]. Configured the local dev stacks accordingly. Used Google Gemini [3] and OpenAI ChatGPT [5] to accelerate prototyping of boilerplate routing patterns. Pushed the baseline project structure to a centralized GitHub repository with custom branch policies. Initialized the localized PostgreSQL engine instance and wrote the structural migrations tracking physical infrastructure parameters (Buildings and Classrooms).
- **Challenges faced:** Encountered unexpected routing anomalies during local PostgreSQL configuration, requiring explicit port forwarding adjustments to confirm network access. Ran into path resolution issues when assigning backend runtime environment variables, creating short connection gaps across runtime boundaries.
- **Progress made:** Completed complete configuration of the unified developmental workspace. Achieved end-to-end repository synchronicity with active master constraints. Formulated initial schema blueprints tracking basic physical university nodes [11].

C. Target for next weekly update

- **Develop the Priority Resolution Engine:** Build the core priority resolution handlers into the relational transaction layer to execute multi-tier conflict checks [11].
- **Implement Core Booking Rules:** Enforce the hard-coded No-Bump Policy to protect already confirmed bookings.
- **Frontend Booking Interfaces:** Create the form inputs for basic room allocation submissions using React component paradigms [7].
- **Authentication & Role Management:** Establish the authentication middleware and map faculty profiles to their deterministic seniority ranks (e.g., Professor = 1, Lecturer = 4) so the conflict engine has data to evaluate [9].

D. Individual contribution

TABLE I
CONTRIBUTION SUMMARY FOR WEEKLY REPORT 1.

Task List	Member Assigned	Comments
Backend setup	Tawhid Hasan, Mohammad Tasvirul Hasan Riyad	Node.js backend API layer configured
Weekly report documentation	Md. Tayasshuk Imam	Managed core document layout structures
Version control / GitHub setup	Md. Tanvir Islam	Git repository and branch policies established
Database configuration	Md. Tayasshuk Imam	PostgreSQL engine initialized; Classroom and Building schemas migrated

II. WEEKLY REPORT 2

A. Follow-up from last week

- **Develop Priority Resolution Engine:** Built initial prioritization logic into the transaction layer [11]. (Completed)
- **Implement Core Booking Rules:** Hard-coded transactional criteria handling basic reservation locks [11]. (Completed)
- **Frontend Booking Interfaces:** Configured entry-level UI submission forms using structured state targets [7]. (Completed)
- **Authentication & Role Management:** Mapped static seniority tables directly within the routing layers [9]. (Completed)

B. Work narrative of the current week

- **Tasks completed:** Programmed the foundational priority validation blocks in Node.js [4] to match structural table lookups. Hard-coded the strict No-Bump engine bounds to block unexpected overrides on established dates. Built modern React input tables to ingest target room metrics [7], parsing variables directly into localized state targets. Consulted OpenAI ChatGPT [5] to optimize complex PostgreSQL query expressions. Evaluated faculty ranking definitions (e.g., Professor = 1, Lecturer = 4) inside state blocks to supply proper comparison trees to transaction queries [9].
- **Challenges faced:** Synchronizing multi-tier database queries with active async wrappers created transient resource locks inside the PostgreSQL driver [6], requiring isolation adjustments [11]. Binding input schema with strict hierarchical validation fields led to short runtime mismatches during complex front-to-back testing passes.
- **Progress made:** Verified complete input-to-storage baseline operational paths. Faculty roles [9], asset availability indexes, and booking rules now validate cleanly before completing localized transaction operations.

C. Target for next weekly update

- **Implement Dashboard Interfaces:** Create centralized dashboard views for both administrative staff and regular faculty to track their active and historical bookings [2].
- **Advanced Search Integration:** Build query filters on the frontend and backend allowing users to search for classrooms based on specific capacity constraints and building locations [1].
- **Automated Notification System:** Configure the foundation for an email or alert service to notify users when a room allocation has been confirmed or rejected.

D. Individual contribution

TABLE II
CONTRIBUTION SUMMARY FOR WEEKLY REPORT 2.

Task List	Member Assigned	Comments
Priority Resolution Engine & Core Rules	Tawhid Hasan, Md. Tayasshuk Imam	Conflict checks and No-Bump Policy enforced in transaction layer
Frontend Booking Interfaces	Md. Tanvir Islam, Mohammad Tasvirul Hasan Riyadh	Built form inputs and managed React states
Authentication & Role Management	Tawhid Hasan	Mapped faculty seniority ranks and established middle-ware

III. WEEKLY REPORT 3

A. Follow-up from last week

- **Implement Dashboard Interfaces:** Developed unique layouts for Admin and Faculty view matrices [2]. (Completed)
- **Advanced Search Integration:** Extended endpoints to accept physical location and asset constraints during search [1]. (Completed)
- **Automated Notification System:** Constructed tracking logic within internal schema mutations to record action traces. (Completed)

B. Work narrative of the current week

- **Tasks completed:** Programmed interactive operational view components for both Faculty and Admin environments [2], [7], using state filters to ensure tailored row rendering based on session roles [9]. Assembled heavily filtered multi-table SQL querying pathways [6] to fetch records matching compound room parameters, spatial tags, and time schedules simultaneously [1]. Utilized Google Gemini [3] to refactor state handling for complex dynamic forms. Added historical record-tracking mechanisms that capture authorization mutations whenever requests are submitted, updated, or shifted by supervisors.
- **Challenges faced:** Parsing combined constraints (overlapping schedule ranges and room equipment rules) within single database executions initially created significant processing bottlenecks [1]. Managing complex nested session components across asynchronous hooks on the React side induced temporary screen-rendering inconsistencies, which required adjusting state cycles.
- **Progress made:** Stabilized interactive contextual workspaces linked across modern analytical table lookups [2]. The application now successfully accepts advanced structural search queries [1] and cleanly distributes proper rendering streams across targeted layout views.

C. Target for next weekly update

- **Integrate Live Email Transmissions:** Connect the backend tracking logs to a live email relay mechanism (e.g., Nodemailer or a dedicated SMTP client) to pass active notices directly to faculty accounts.
- **Build Analytics Graphs:** Introduce visual data blocks or charting libraries on the Admin panel using Recharts [8] to summarize daily room occupancy and usage densities [2].
- **Optimize Multi-tier Lock Boundaries:** Perform load and strain testing across the reservation transaction parameters to prevent race conditions during heavy enrollment peaks [11].

D. Individual contribution

TABLE III
CONTRIBUTION SUMMARY FOR WEEKLY REPORT 3.

Task List	Member Assigned	Comments
Dashboard Interface Generation	Mohammad Tasvirul Hasan Riyad, Md. Tanvir Islam	Assembled responsive grid structures and handled contextual rendering elements
Dynamic Query Parameters	Tawhid Hasan	Built optimal indexed queries to evaluate room assets and physical availability
State Logs	Md. Tayasshuk Imam	Initialized event tables to monitor booking mutations and record actions

IV. WEEKLY REPORT 4

A. Follow-up from last week

- **Integrate Live Email Transmissions:** Connected backend tracking logs to a live Nodemailer SMTP service to deliver real-time booking status notifications to faculty accounts. (Completed)
- **Build Analytics Graphs:** Integrated interactive visualization components into the Admin panel using Recharts [8] to summarize daily room occupancy and usage densities [2]. (Completed)
- **Optimize Multi-tier Lock Boundaries:** Refactored PostgreSQL isolation levels [6] and transactional queries to maintain deterministic concurrency handling and prevent race conditions during heavy enrollment peaks [11]. (Completed)

B. Work narrative of the current week

- **Tasks completed:** Configured a Nodemailer transport client integrated with Node.js background event triggers [4] to fire asynchronous transactional email alerts for confirmation, cancellation, and modification events. Implemented graphical dashboard analytics utilizing Recharts [8] on the admin interface, rendering real-time capacity metrics, building utilization heatmaps, and hourly allocation trends [2]. Performed concurrent transaction load testing against the priority reservation engine; updated execution paths to utilize strict row-level locking (`SELECT . . . FOR UPDATE`) within explicit PostgreSQL serializable transactions [6] to prevent race conditions [11].
- **Challenges faced:** Asynchronous email delivery streams initially caused request processing delays during database commits; resolved this by decoupling notification triggers into background job queues. Encountered intermittent deadlocks under simulated concurrent booking requests targeting identical room/time parameters; mitigated these by enforcing strict table access ordering across transaction logic [11].
- **Progress made:** Successfully stabilized the high-concurrency booking pipeline while adding comprehensive administrative analytical insights [2]. Real-time email delivery is now fully operational, and system transaction boundaries safely handle simultaneous reservation attempts without race conditions or data corruption [11].

C. Target for next weekly update

- **Comprehensive Stress & Penetration Testing:** Conduct full-scale end-to-end performance benchmarks and security audits (role elevation, SQL injection, input sanitization) [10].
- **Final System Refinement & Polish:** Address minor UI/UX anomalies, standardize error messaging across API endpoints, and refine responsive dashboard view layouts [2].
- **Documentation & Deployment Preparation:** Draft final technical documentation, finalize setup scripts, and configure production container environments for deployment.

D. Individual contribution

TABLE IV
CONTRIBUTION SUMMARY FOR WEEKLY REPORT 4.

Task List	Member Assigned	Comments
Email Relay & Transmissions	Md. Tayasshuk Imam	Integrated Nodemailer background handlers for real-time notification alerts
Analytics & Graphical Dashboards	Mohammad Tasvirul Hasan Riyad	Formulated chart layouts using Recharts to present room occupancy and usage metrics
Lock Boundary Optimization	Tawhid Hasan	Implemented row-level locking and optimized transaction isolation levels to eliminate race conditions
Load Testing & Validation	Md. Tanvir Islam	Executed concurrency scripts and benchmarked database connection pool stability

V. WEEKLY REPORT 5

A. Follow-up from last week

- **Comprehensive Stress & Penetration Testing:** Executed full-scale security testing suites covering SQL injection, role privilege boundaries, and endpoint input sanitization [10]. (Completed)
- **Final System Refinement & Polish:** Addressed responsive layout bugs across analytics dashboards [8] and standardized API error messaging [4]. (Completed)
- **Documentation & Deployment Preparation:** Drafted operational system documentation, setup scripts, and finalized production Docker deployment configurations. (Completed)

B. Work narrative of the current week

- **Tasks completed:** Conducted comprehensive penetration and load testing against active endpoints to ensure resilience against vulnerability exploits [10]. Applied final UI/UX refinements to the administrative and faculty dashboards [7], ensuring responsive consistency across viewing platforms [2]. Leveraged OpenAI ChatGPT [5] and Google Gemini [3] for code review and automated unit test case generation. Completed Docker containerization setup for backend services and PostgreSQL instances [6], verifying local production-ready builds.
- **Challenges faced:** Mitigated edge-case JWT authorization flaws during rapid token refreshing cycles under high concurrency. Standardized container health-check execution parameters to prevent cold-start delays across service boundaries.
- **Progress made:** Successfully concluded all core development and system integration phases. The system is hardened against concurrency conflicts [11] and security vulnerabilities [10], fully containerized, and prepared for final evaluation.

C. Target for final submission & presentation

- **Final Project Report Compilation:** Consolidate architectural diagrams, schema models, performance metrics, and weekly narratives into the comprehensive Final CSE299 Report.
- **Final System Demonstration & Defense:** Prepare live software demonstration and present end-to-end functionality before the evaluation panel.

D. Individual contribution

TABLE V
CONTRIBUTION SUMMARY FOR WEEKLY REPORT 5.

Task List	Member Assigned	Comments
Security Audits & Penetration Testing	Md. Tanvir Islam	Executed security scans, SQL injection resistance testing, and endpoint validation
UI/UX Polish & Interface Standardization	Mohammad Tasvirul Hasan Riyad	Normalized REST response structures and finalized responsive frontend dashboard views
Docker Build & Deployment Configuration	Tawhid Hasan	Built production Docker images and configured multi-container orchestrations
Weekly & Final Report Compilation	Md. Tayasshuk Imam	Synthesized final weekly narratives, verified layout compliance, and initiated final project report drafting

REFERENCES

- [1] R. Baeza-Yates and B. Ribeiro-Neto. *Modern Information Retrieval: The Concepts and Technology Behind Search*. Addison-Wesley, 2nd edition, 2011.
- [2] S. Few. *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*. Analytics Press, 2nd edition, 2013.
- [3] Google. Google Gemini AI Model Documentation. Available: <https://deepmind.google/technologies/gemini/>. Accessed: 2026.
- [4] Node.js Foundation. Node.js v20 LTS Documentation. Available: <https://nodejs.org/docs/>. Accessed: 2026.
- [5] OpenAI. ChatGPT and GPT-4 Technical Documentation. Available: <https://platform.openai.com/docs/>. Accessed: 2026.
- [6] PostgreSQL Global Development Group. PostgreSQL 16 Documentation: Concurrency Control and Isolation. Available: <https://www.postgresql.org/docs/>. Accessed: 2026.
- [7] React Documentation. Building User Interfaces with React 19. Available: <https://react.dev/>. Accessed: 2026.
- [8] Recharts Group. Recharts: A Redefined Chart Library Built with React and D3. Available: <https://recharts.org/>. Accessed: 2026.
- [9] R. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.
- [10] A. Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, 2014.
- [11] A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill, 7th edition, 2020.